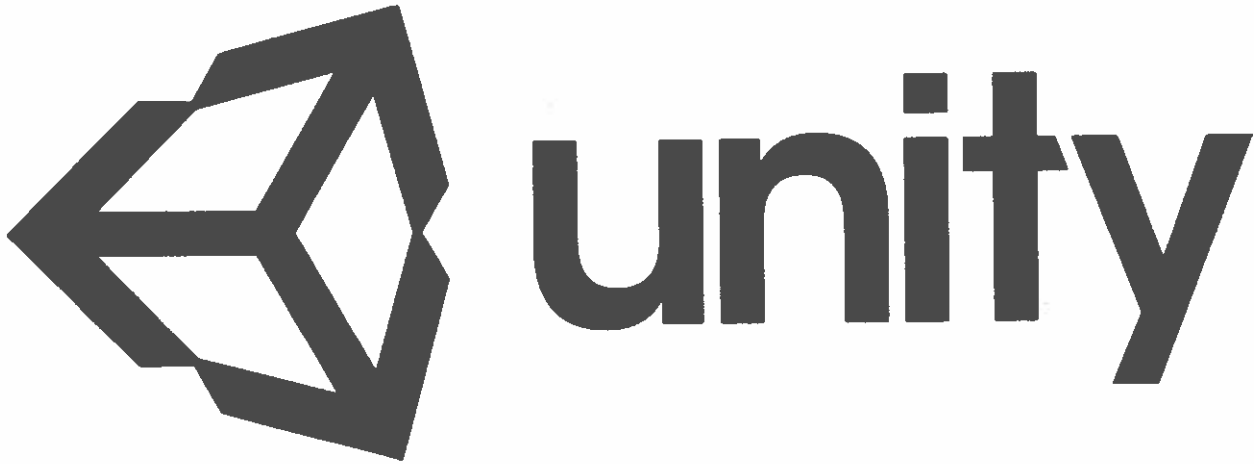


Senior Project Defense



College Football Tower Defense Game

Contact Information

David McLaughlin
2267 Sandy Point Ln.
Mt. Pleasant, SC 29466
Djmclaughlin7692@gmail.com

Under the Supervision of

Proffessor Gary Underwood
Professor of Computer Science/Advisor
Charleston Southern University
gunderwood@csuniv.edu

School Information

Charleston Southern University
9200, University Blvd. North Charleston, SC 29406
(843) 863-7000

Table of Contents

Introduction.....	1
Purpose and Background Information.....	1
Unity.....	2
Basic Timeline of Project.....	3
Program Details.....	4
How to Play.....	6
Towers.....	6
Enemies.....	7
Game Overview.....	7
Objects on Screen and Corresponding Scripts.....	10
Other objects and Corresponding Scripts.....	12
Scripts and Descriptions.....	14
Future Plans.....	16
Sources.....	17
Screen Shots of Scripts	

Introduction

Purpose and Background Information:

I came to Charleston Southern University as an Athletic Training major never knowing that I would decide to change to a completely unrelated field. The transition to Computer Science was a tough transition, but if I would have had the chance to do it all over again, I would have most certainly chosen this field again. Before becoming a Computer Science major I knew almost nothing about computers. I cannot explain the pride I have gained by gaining the knowledge of being able to program, set up networks, understand the inner working of computers and many more subject. But the greatest skill I have obtained from this major is the ability to learn. I chose to do a game because I wanted to something I could have never thought I would try to do. Creating a game is a very different endeavor than I have ever tried. I have never been very enthusiastic about gaming, but I have always been interested in the inner workings of them and the process of their creation.

In the fall semester of 2014 I chose to make a tower defense game. Under the advisement of Professor Underwood, this project was seen fit as a senior project level task. Midway through the spring semester of 2015 I had the code architecture of my project and started learning OpenGL programming.

I began this project planning to use OpenGL and the C++ language. I chose to use OpenGL over DirectX, even though for what I needed both would have been suitable. I chose OpenGL because OpenGL has a very strong and loyal following and there was a lot more resources for beginner game developers than there was for DirectX. I jumped into OpenGL

by doing many tutorials and also buying books on OpenGL. While doing my research in the summer of 2015 I discovered Unity.

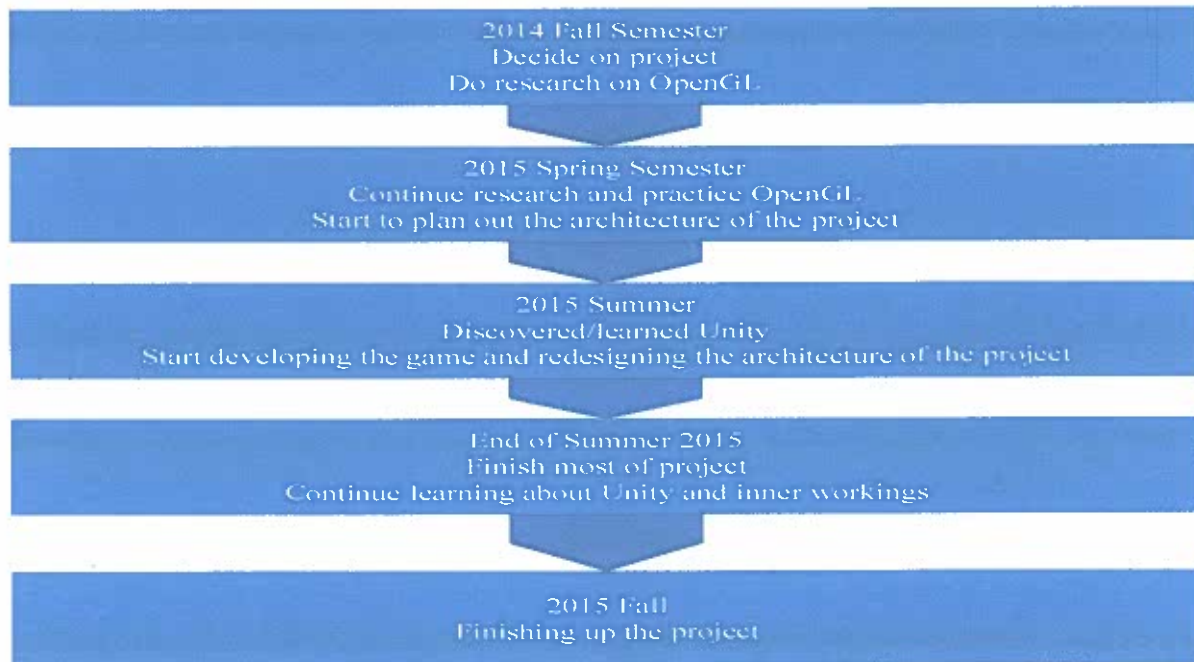
Unity

Unity is cross-platform Game Engine used to make games for consoles, PCs and websites. Unity developers either use C# or JS to develop games. Learning to use Unity SDK was certainly a challenge but it was made easier with Unity itself having many tutorials and an open community of developers which were both very helpful. Having learned how to make and manipulate game objects within Unity has given me a deeper appreciation of game development in itself.

All objects in the game are gameobjects. Gameobjects are just empty containers without any properties and do not do anything. To make gameobjects active parts of the game one must add properties to the gameobject. These properties could be colliders so that other gameobjects can be effected by it, or the property could be a script that cases the gameobject to react to its environment.

I started developing my game in the summer of 2015 with Unity. By the end of the summer I had a strong understanding of how Unity worked and how the architecture of my game would function.

Basic Timeline of the Project



Program Details

Challenges

Making my first game has come with many challenges. These challenges ranged from me using the skills and knowledge to do a much different project than I have done before to learning how to use a game engine.

Not having the knowledge about Unity caused me to have many problems with the way that I had planned to do things. Many times I would spend hours working on a problem, when I would finally try a different way; a way that I would not have tried before. I would find that the problem would be fixed in less than an hour. Learning how Unity works has challenged me to work with different ways that I would never have thought to do in the past. An example would be the creation of the explosions from the kicker's footballs. I must have tried five different ways to create the explosions, affect the enemies with the explosions and also destroying the explosion game object.

Another challenge was making the game look good. I am most certainly not a skilled artist so trying to get the pictures in my head on the screen was a very big obstacle for me. With a lot of patience and MS paint I was able to get the images as close to what I wanted as possible.

As a programmer I have never been able to follow one of the main rules of programming: backing up my code. Even on my very big projects, I have never been good at remembering that in a split second everything could be ruined. During the end of the creation of the first game, I was going to delete a script I had planned on using but ended up figuring out a different way. What I thought I had selected before deleting was this one unused script. What I had selected was my entire scripts folder. This happened four weeks before the end of the semester. With

many sleepless nights though I was able to redo all of my code. I have finally learned my lesson and now backup all my code at least three times.

How to Play

Towers

The towers are based on American football positions. The towers include a Quarter Back, Kicker and Referee. The towers are placed on the game plane and wait until an enemy enters a preselected radius. When the tower sees an enemy it will target that enemy and throw its specific projectile at the enemy. Each tower has various attributes including cost, attack rate, projectile speed, attack radius and damage. Each tower has the BasicTower.cs script connected to it that handles all of these parts.

The first tower is the “Quarter Back” tower. It is the least expensive tower out of the three towers. It has a mid-range attack radius, a moderate damage attack, quick attack rate and the speed of its projectile is fast. The QB’s projectile is just a normal football that does not do anything other than subtract health from the enemy that the projectile collided with.

The second tower is the “Referee” tower. It is the second most expensive tower. It has a mid-range attack radius, low damage attack, medium attack speed and the speed of its projectile is fast. The Ref’s projectile is the referee’s flag. When the flag collides with an enemy, it will slow that enemy’s speed by fifty percent. While the actual damage done by the Referee is low, the impact of slowing the enemy down gives the player more time to destroy the enemy. The “Refsflag.cs” script is connected to the referee’s projectile. This script handles slowing down the enemy that the flag collides with.

The last tower is the “Kicker” tower. It is the most expensive tower in the game. It has a long range attack radius, high damage attack, slow attack rate and the speed of its projectile is slow. The Kicker’s projectile is the Kicker’s football. When the football collides with an enemy,

it explodes. The explosion lasts for only a second. If an enemy collides with the explosion the enemy will be hurt again. The actual creation of the explosion is handled by the enemy that the kicker's projectile collides with. The kicker's football is controlled by the script, "ExplosionHandler.cs". This script controls how long the explosion lasts.

Enemies

The enemies in the game are based on collegiate mascots. The mascots are spawned on the game's plane and follow a path set up by way points. The enemy moves across the map by moving straight from waypoint to waypoint. Each enemy has a specified speed, money worth, health and damage. Speed is how quickly they move from waypoint to waypoint across the map. The worth of the enemy is how much money the player gets if the player destroys the enemy. Health is the amount of damage an enemy can sustain before being destroyed. Damage of the enemy is the amount of health that the enemy will take away from the player if they make it to the end of the map. The scripts BasicEnemy.cs, EnemyMovement.cs and OnEnemyDestroy.cs handle all of these functions of the enemies.

The AElephant is the slowest of all of the enemies, but has the most health and causes the most damage. The AElephant also has the highest money aspect out of the enemies.

The CTiger is the second fastest enemy in the game. The CTiger has the second most health and damage and also has the second highest money aspect out of the enemies.

The GCock is the fastest out of all of the enemies. Though they are the fastest enemies they have the least amount of health. Being easy to destroy, the GCock has the lowest money aspect out of the enemies.

Game Overview

The game is played by a single human player. The player has a choice of three levels to choose from. The player can place towers on the plane of the game if they have enough money to get the specified towers. When the player is ready for the enemies, the player clicks the PlayButton, which will start the level's wave of enemies. Each level has a specified number and combination of different enemies.

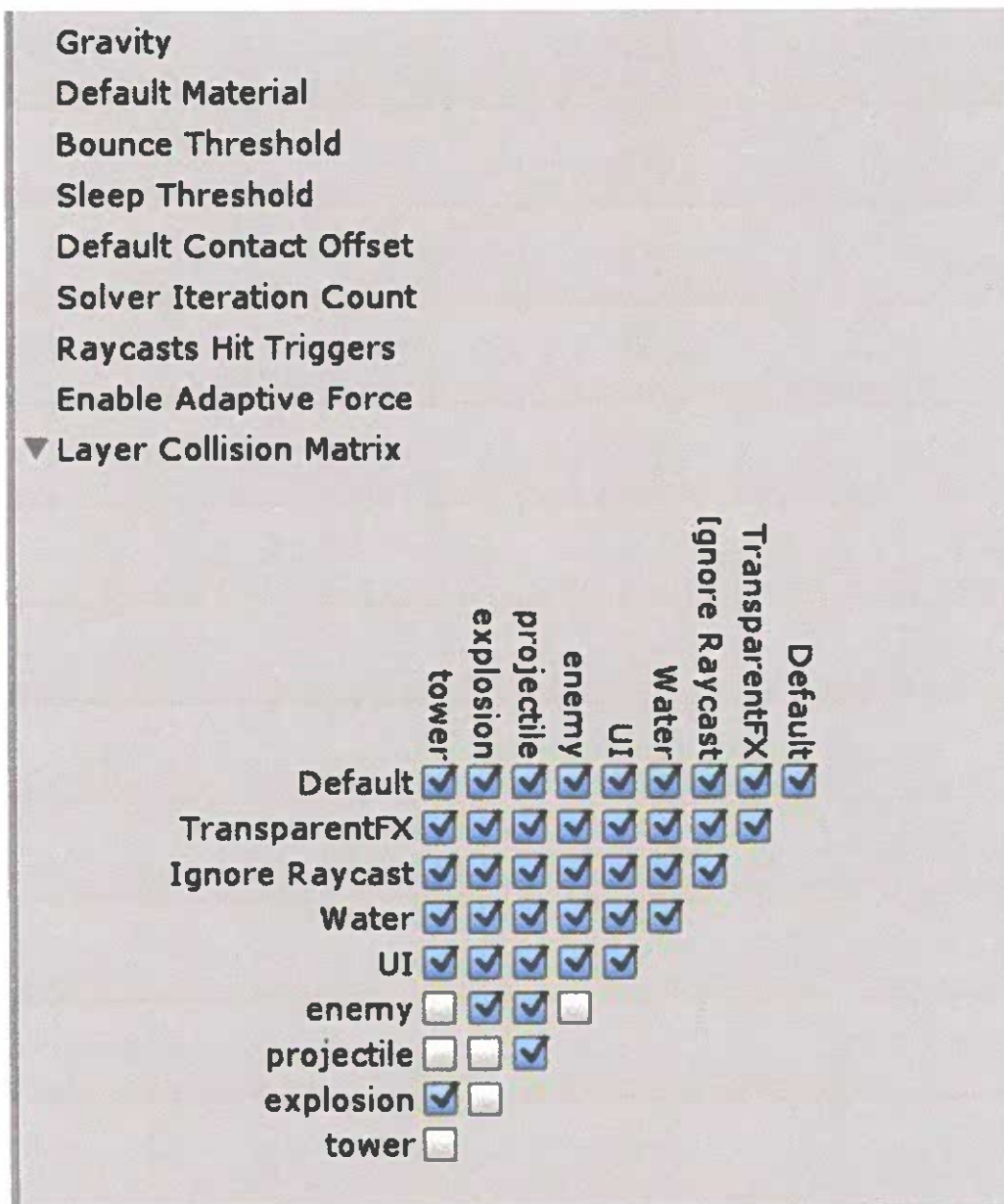
The enemies are spawned by the mascotspawner that has the MascotSpawner.cs script connected to it that spawns the enemies on the first waypoint. The enemies move through the map by moving to waypoints in the set order based on the map.

The player has to try to stop the enemies from making it to the end zone of the map by placing towers that shoot projectiles at the enemies to destroy them around the map. If the player makes it through all of the levels, then the player wins the game and can play again. If the player runs out of lives before making it to the end of the levels, then the player loses.

Tags and Layers

Gameobjects can have properties connected to them called tags layers. A tag is a property that is just a word to identify that object. The tag on the enemies is "enemy". The enemies' tag is used by the towers to identify enemies in their radius of view to target the proper gameobjects. The tags on the projectiles and explosion is used by the enemies to determine how much of their life to be subtract depending on the particular projectile. Layers in the project are used in a similar way as tags. Layers in the game are used to determine what gameobjects will collide with each other. By settling the projectiles, towers, and enemies to certain layers. I have made it so that projectiles will not hit the explosion but will hit and affect enemies but not towers. I also set

that towers and enemies will not collide so that towers will not affect the movement of the enemies. I set these settings up in the physics layer collision matrix of the game.



Objects on Screen and Corresponding Scripts:

1. BackButton

- a. SelectButtonByClick.cs

2. HealthDescriptionText

3. HealthText

4. KickerSelectBlock

- a. SelectButtonByClick.cs

5. KickerSelectText

6. LevelDescriptionText

7. LevelText

8. Main Camera

- a. DetectClicksOnScreen.cs
- b. ShowButtonSelectandBackButton.cs

9. Mascotspawn

- a. MascotSpawner.cs

10. MoneyDescriptionText

11. MoneyText

12. Plane

- a. CreateTowerWhenClick.cs
- b. LevelTracker.cs
- c. MoneyTracker.cs
- d. HealthTracker.cs

13. PlayButton

- a. SelectButtonByClick.cs

14. QBSelectBlock

- a. SelectButtonByClick.cs

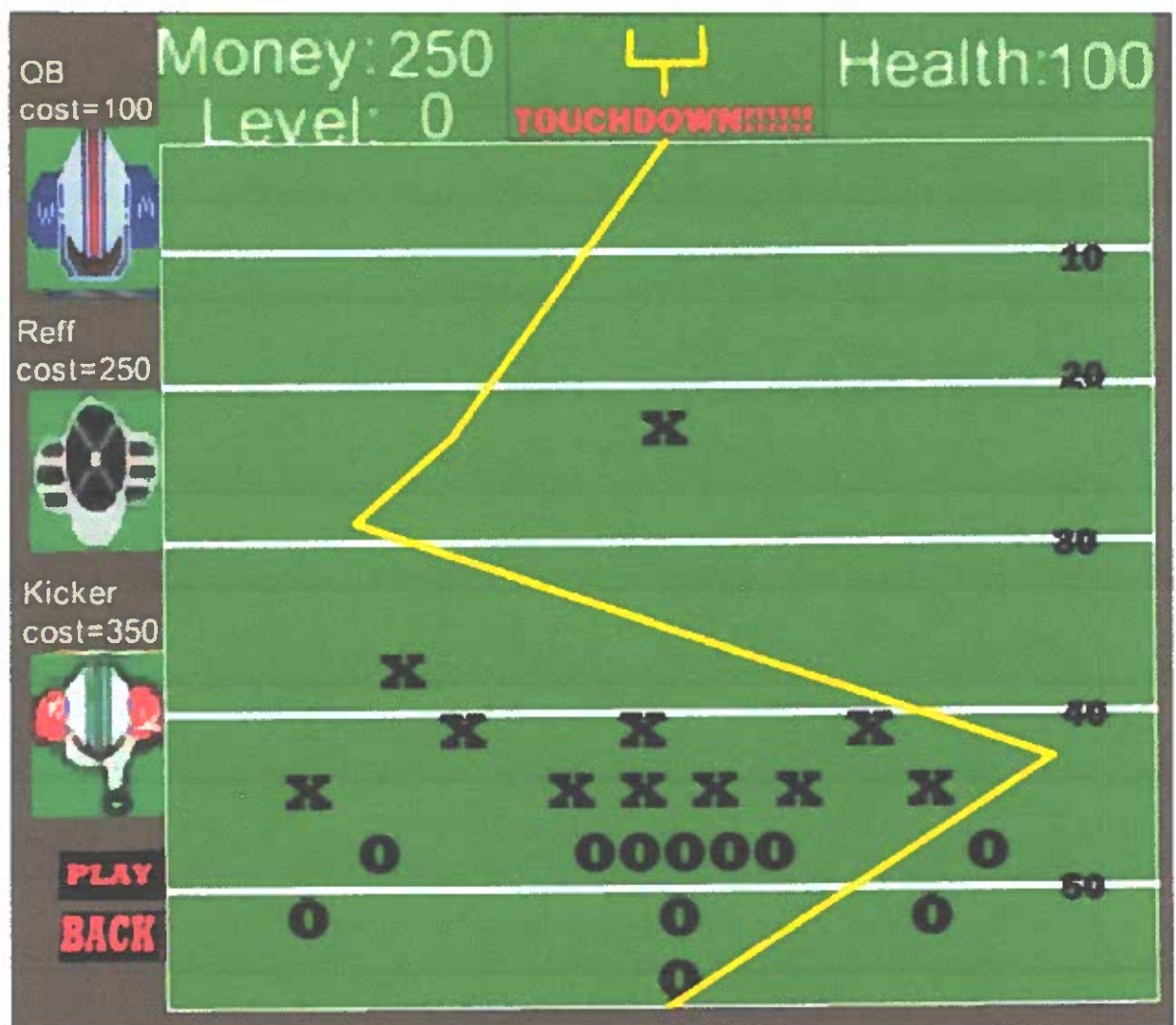
15. QBSelectText

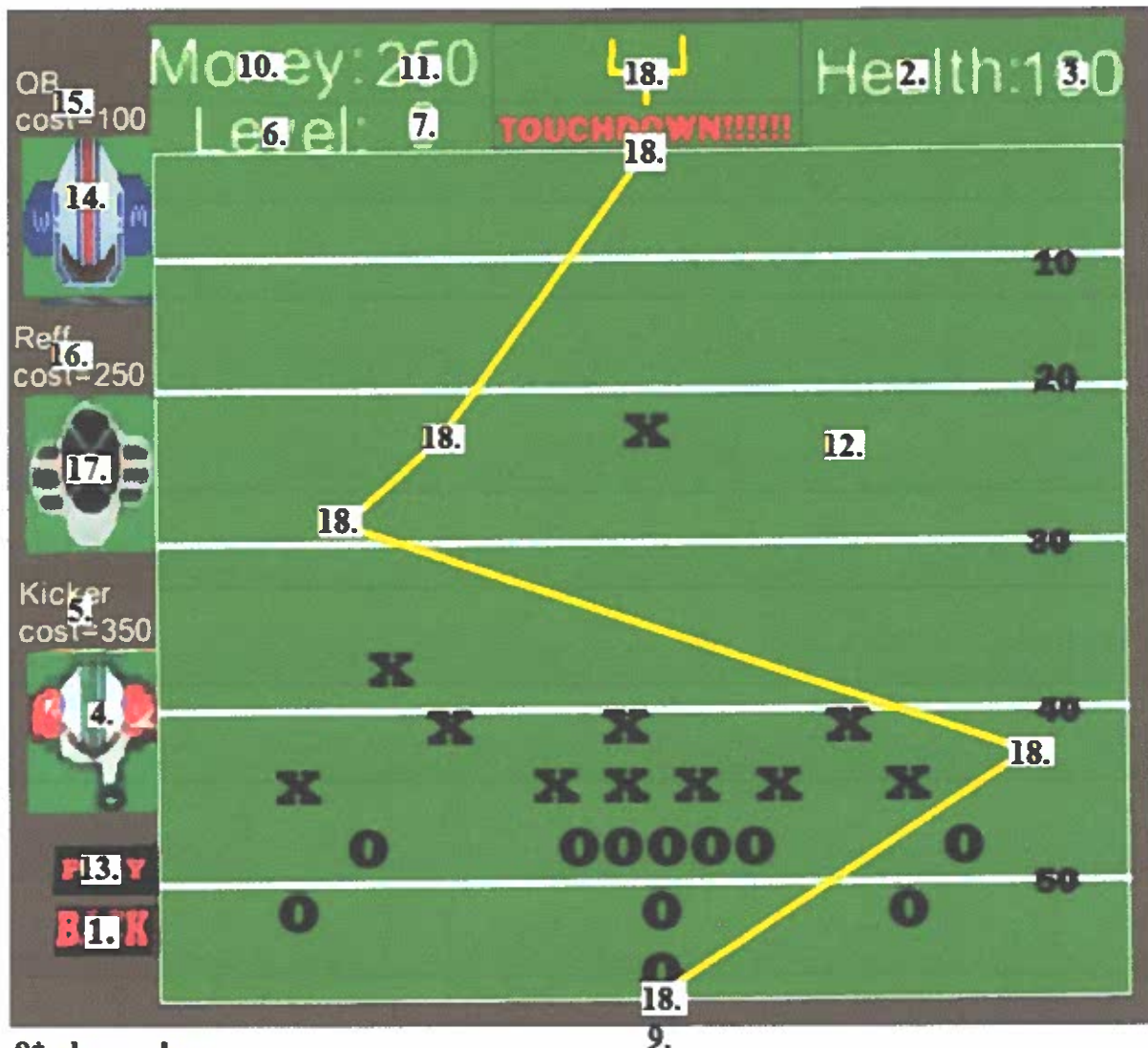
16. ReffsSelectText

17. ReffsSelectTextBlock

- a. SelectButtonByClick.cs

18. Start, FirstTurn-FourthTurn, EndSpot





8* above plane
looking down

Other objects and Corresponding Scripts

1. Enemies

- a. BasicEnemy.cs
- b. EnemyMovement.cs
- c. OnenemyDestroy.cs

2. Towers

- a. BasicTower.cs

3. Football

- a. DestroyLeftOverFootballs.cs

4. KickersBall

- a. DestroyLeftOverFootballs.cs

5. RefsFlag

- a. DestroyLeftOverFootballs.cs
 - b. TheResflag.cs
- 6. **SphereOfExplosion**
 - a. ExplosionHandler.cs

Scripts and Descriptions:

1. BasicEnemy.cs

- Handles when the enemy gets hit by projectiles. Also creates the explosion when hit by the Kicker tower.

2. BasicTower.cs

- Handles finding enemies within the tower's set radius and throwing projectiles at the targeted enemies.

3. ChangeScene.cs

- Changes to different scene in the game when sent name of new scene. Also changes scene when the player wins or loses.

4. CreateTowerWhenClick.cs

- Makes sure that the player has enough money to make tower, if so the tower is created on the plane where the player clicks

5. DestroyLeftOverFootballs.cs

- Destroys the projectiles after 5 seconds so missed projectiles are not just floating around

6. DetectClicksOnScreen.cs

- Sees and recognizes clicks from the player in the view of the camera

7. EnemyMovement.cs

- Is sent the waypoints of the map and controls

8. Explosion.cs

- Destroys the explosion game object after a certain amount of time.

9. HealthTracker.cs

- Controls the Health of the player.

10. LevelTracker.cs

- Tracks the current level the player is on.

11. MascotSpawner.cs

- Has all of the levels as arrays. Based on the current level starts the instantiation of the corresponding level array.

12. MoneyTracker.cs

- Keeps track of the player's current money during the game.

13. OnenemyDestroy.cs

- Controls when the enemy is destroyed by a tower the corresponding money is added to the player's money.

14. SelectButtonByClick.cs

- Receives the object clicked and sends it to ShowButtonSelectandBackButton.cs so it can show which tower was selected and CreateTowerWhenclick.cs to create the selected tower.

15. ShowButtonSelectandBackButton.cs

- Handles showing which tower has been selected by spinning the tower icon.

16. TheRefsflag.cs

- When the referee's flag hits an enemy this halves the enemy's speed.

Future Plans

If I would have had more time, there would be a few things I would have liked to add. I would have liked to add to the game the ability to affect the towers by being able to upgrade them or sell them. The next thing I would have wanted to learn about in Unity is the animation aspects, which I never was able to get to because of running into complications and setbacks. I want to learn how to make my own game objects and stepping through how they would react to their environments.

Things I could do now would include adding more towers and adding more maps. To add a new map I could duplicate the scene and change where the way points are. To add more towers I would just duplicate a tower and change some of their specs. Adding a new enemy would be easy since one could just duplicate a different enemy, change the image and the specs.

To add abilities to the player like upgrading and selling the towers that have been made on the screen would be harder than the other plans in a coding perspective. I would have to change the architecture of the towers code so that it could be easily manipulated by the player.

Sources:

<http://docs.unity3d.com/ScriptReference/Vector3.html>

<https://unity3d.com/learn/tutorials/modules/beginner/scripting/invoke?playlist=17117>

<http://docs.unity3d.com/ScriptReference/Input.GetMouseButtonDown.html>

<http://docs.unity3d.com/ScriptReference/Collider.OnCollisionEnter.html>

<http://docs.unity3d.com/ScriptReference/Transform.LookAt.html>

<http://answers.unity3d.com/questions/132255/how-to-switch-between-scene.html>

<https://unity3d.com/learn/tutorials/projects/stealth/enemy-ai>

<http://classroomclipart.com/>

```
using UnityEngine;
using System.Collections;

public class BasicEnemy : MonoBehaviour {
    public int health = 10;

    public GameObject explosions;

    //Handles when a football or any type of projectile
    //subtracts a certain amount of health based on the projectile
    //lastly destroys the projectile
    void OnCollisionEnter(Collision collision){
        if(collision.collider.tag == "flag")
        {
            health--;
            Destroy(collision.collider.gameObject);
        }
        if(collision.collider.tag == "football")
        {
            health-=2;
            Destroy(collision.collider.gameObject);
        }
        if(collision.collider.tag == "kickersfootball")
        {
            health-=5;
            Destroy(collision.collider.gameObject);
            //when the ball hits the explosion will happen and Explosion.cs will handle the
            lifespan of the explosion
            Instantiate(explosions,collision.collider.transform.position, explosions.
            transform.rotation);
        }
        if (collision.collider.tag == "explosion")
        {
            health--;
        }
        if(health <= 0)
        {
            Destroy(gameObject);
        }
    }
}
```

```

using UnityEngine;
using System.Collections;

public class BasicTower : MonoBehaviour {
    public GameObject Football;
    public GameObject Tower;
    public float speedOfFootball=1.0f;
    public float rateOfThrow=1.0f;
    public float towerRaidius= 5.0f;
    // Use this for initialization
    void Start () {
        //start the function throwFootball with no delay, then have it happen ever time at
        rateOfThrow
        InvokeRepeating ("throwFootball", 0.0f, rateOfThrow);
    }
    void throwFootball(){
        GameObject target=null;
        //makes the shooting range of the tower based the specks of the tower
        foreach(Collider col in Physics.OverlapSphere(transform.position, towerRaidius)){
            if(col.tag=="enemy"){
                target=col.gameObject;
                break;
            }
        }
        //see an enemy since the enemy has an enemy tag on it
        if(target!=null){
            GameObject newFootball = Instantiate(Football, transform.position, Football.
            transform.rotation) as GameObject;
            Tower.transform.LookAt (target.gameObject.transform.position);//turn the tower
            newFootball.GetComponent<Rigidbody>().AddForce ((target.transform.position -
            transform.position).normalized * speedOfFootball);
        }
    }
}

```

```
using UnityEngine;
using System.Collections;

public class ChangeScene : MonoBehaviour {

    public static bool won=false;
    public static bool lost=false;
    public void changeToDifferentScene(string SceneName) {
        Application.LoadLevel(SceneName);
    }
    //checks if the player has made it to the end of the game or ran out of lives
    public void Update(){
        if (won == true) {
            Application.LoadLevel ("WinScene");
            won=false;
        }
        if (lost == true) {
            Application.LoadLevel("LosingScene");
            lost=false;
        }
    }
}
```

```

using UnityEngine;
using System.Collections;

public class CreateTowerWhenClick : MonoBehaviour {
    public GameObject TowerIcon1;
    public GameObject TowerIcon2;
    public GameObject TowerIcon3;
    public GameObject Tower1;
    public GameObject Tower2;
    public GameObject Tower3;
    public GameObject TheChosenTower;

    public float tower1Cost=100.0f;
    public float tower2Cost=250.0f;
    public float tower3Cost=400.0f;

    //sets theChosenTower to the Clicked icon
    void getSelectedTower(GameObject selectedtower){
        TheChosenTower = selectedtower;
    }
    //gets the vector3 position (3D position)
    void Clicked(Vector3 position)
    {
        //Debug.Log (position);
        //having them spawn .25 up will put the towers right on the plane
        //Compares theChosenTower to the towericons to see what tower to place down.
        //When correct tower is found then the next time the player clicks on the plane
        //the selected tower will be created on that position. This happens if
        //the player has enough money, checked by MoneyTracker's variable money.
        //If they do have enough the cost of the tower is subtracted from MoneyTracker's
        variable money.
        if (TheChosenTower == TowerIcon1) {
            if(MoneyTracker.money>=tower1Cost){
                Instantiate(Tower1,position + Vector3.up*0.25f,Tower1.transform.rotation);
                MoneyTracker.money-=tower1Cost;
            }
        }
        if (TheChosenTower == TowerIcon2) {
            if(MoneyTracker.money>=tower2Cost){
                Instantiate(Tower2,position + Vector3.up*0.25f,Tower2.transform.rotation);
                MoneyTracker.money-=tower2Cost;
            }
        }
        if (TheChosenTower == TowerIcon3) {
            if(MoneyTracker.money>=tower3Cost){
                Instantiate(Tower3,position + Vector3.up*0.25f,Tower3.transform.rotation);
                MoneyTracker.money-=tower3Cost;
            }
        }
    }
}

```



```
using UnityEngine;
using System.Collections;

public class DestroyLeftOverFootballs : MonoBehaviour {
    //make sure there aren't footballs everywhere off the map
    //footballs are destroyed after 5 seconds
    public GameObject football;

    // Update is called once per frame
    void Update () {
        Destroy(football, 5.0f);
    }
}
```

```
using UnityEngine;
using System.Collections;

public class DetectClicksOnScreen : MonoBehaviour {
    public Camera detectionCamera;
    //This is the actual camera we reference in the update loop, set in Start()
    private Camera thecamera;

    // Update is called once per frame
    void Update ()
    {
        Ray theRay; //A ray is an infinite line starting at origin and going in some
        direction.
        RaycastHit hit;//the spot where a click will happen

        if(Input.GetMouseButtonDown(0)) //Checks left mouse button has been pressed
        {
            //Set up our ray from screen to scene
            theRay = detectionCamera.ScreenPointToRay(Input.mousePosition);
            //If hit
            if(Physics.Raycast (theRay, out hit, Mathf.Infinity))
            {
                hit.transform.gameObject.SendMessage("Clicked", hit.point, SendMessageOptions
                .DontRequireReceiver);
            }
        }
    }
}
```

```

using UnityEngine;
using System.Collections;

public class EnemyMovement : MonoBehaviour {
    public GameObject[] waypoints;
    public GameObject theMascot;

    public float speed = 1.0f; // speed of the basic enemy, can be changed based on enemy
    public float originalSpeed = 1.0f; // this is here for the ref's flag to see if the tower
    had already been slowed down
    public float Money = 25.0f; // this is here to make sure when the enemy is destroyed by
    making it all the way the player doesn't get money like if the enemy is destroyed but
    being shot
    public float enemyDamage = 1.0f; // the amount of live that the enemy takes away when they
    make it the end of the map

    private int spotintWayPoints = 0;

    private Vector3 movementDirection;
    // where the mascot should start
    Vector3 temp = new Vector3(239.0f, 130.0f, 5.0f);

    void Start()
    {
        // set the mascot's starting spot
        theMascot.transform.position = temp;
        movementDirection = (waypoints[spotintWayPoints].transform.position - transform.
        position).normalized;
    }
    // Update is called once per frame
    void Update ()
    {
        // movement of the enemy
        transform.position += movementDirection * speed * Time.deltaTime;
        if (speed != originalSpeed) {
            // creates yellow halo around the enemy to show that it
            // is slowed down
            (theMascot.GetComponent("Halo") as Behaviour).enabled = true;
        }
    }
    // triggered by hitting waypoint
    void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.name == waypoints[spotintWayPoints].name)
        {
            transform.position = waypoints[spotintWayPoints].transform.position;
            spotintWayPoints++;
            if (spotintWayPoints >= waypoints.Length) // makes to the end without being killed
            {
                // money is subtracted to offset the the destroying
                MoneyTracker.money -= Money;
                HealthTracker.health -= enemyDamage;
            }
        }
    }
}

```

```
        Destroy(gameObject);
    }
    else
    {
        movementDirection = (wayPoints[spotintWayPoints].transform.position -
            transform.position).normalized;
    }
}

//gets the array of wayPoints from MascotSpawner
void setTheMascotWay(GameObject[] inputPathPoints)
{
    wayPoints = inputPathPoints;
}
}
```

```
using UnityEngine;
using System.Collections;

public class ExplosionHandler : MonoBehaviour {
    public GameObject TheActualExplosion;

    public static Vector3 collisionSpot;
    //the explosion will be destroyed after a second
    void Update () {
        Destroy(TheActualExplosion, 1.0f);
    }
}
```

```
using UnityEngine;
using System.Collections;

public class HealthTracker : MonoBehaviour {

    public int startHealth = 100;
    public GameObject Healthtext;

    public static float health=0;

    // Use this for initialization
    void Start ()
    {
        health = startHealth;
    }
    void Update ()
    {
        Healthtext.GetComponent<TextMesh>().text = health.ToString();
        if (health <= 0) {
            ChangeScene.lost=true;
        }
    }
}
```

```
using UnityEngine;
using System.Collections;

public class LevelTracker : MonoBehaviour {

    public int startLevel = 1;
    public GameObject Leveltext;

    public static float CurrentLevel=0;

    // Use this for initialization
    void Start ()
    {
        CurrentLevel = startLevel;
    }
    void Update()
    {
        Leveltext.GetComponent<TextMesh>().text = CurrentLevel.ToString();
        if (CurrentLevel > 10) {
            ChangeScene.won=true;
        }
    }
}
```

```

using UnityEngine;
using System.Collections;

public class MascotSpawner : MonoBehaviour {
    public GameObject[] pathPoints;
    //all of the level arrays which have the list of enemies
    public GameObject[] spawnListLevel1;
    public GameObject[] spawnListLevel2;
    public GameObject[] spawnListLevel3;
    public GameObject[] spawnListLevel4;
    public GameObject[] spawnListLevel5;
    public GameObject[] spawnListLevel6;
    public GameObject[] spawnListLevel7;
    public GameObject[] spawnListLevel8;
    public GameObject[] spawnListLevel9;
    public GameObject[] spawnListLevel10;

    public float spawnTime;//the time between spawns

    public static bool Play=false;

    private int spotInspawnList = 0;

    public int level=0;

    void BeginTheRelease(bool givenBool){
        bool playButton=givenBool;

        if (playButton == true) {
            InvokeRepeating ("PlayCurrentRound", spotInspawnList, spawnTime);
            Play=true;
        }
        Play = false;
        level += 1;
        LevelTracker.CurrentLevel = level;
        spotInspawnList = 0;
    }

    void PlayCurrentRound()
    {
        switch(level){
            case 1:
                //Spawn next enemy in spawnlist
                //starts the spawnlist and the position of the start
                GameObject referencel = Instantiate (spawnListLevel1 [spotInspawnList], transform
                .position, Quaternion.identity) as GameObject;
                //sets the next spawn to the next spot in spawnlist
                spotInspawnList++;
                //when past the end of the spawnlist stop the invoking
                if (spotInspawnList >= spawnListLevel1.Length) {
                    CancelInvoke ();
                }
                //Set enemy path information

```



```
reference1.SendMessage ("setTheMascotWay", pathPoints);
break;
case 2:
    GameObject reference2 = Instantiate (spawnListLevel2 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel2.Length) {
        CancelInvoke ();
    }
    reference2.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 3:
    GameObject reference3 = Instantiate (spawnListLevel3 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel3.Length) {
        CancelInvoke ();
    }
    reference3.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 4:
    GameObject reference4 = Instantiate (spawnListLevel4 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel4.Length) {
        CancelInvoke ();
    }
    reference4.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 5:
    GameObject reference5 = Instantiate (spawnListLevel5 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel5.Length) {
        CancelInvoke ();
    }
    reference5.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 6:
    GameObject reference6 = Instantiate (spawnListLevel6 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel6.Length) {
        CancelInvoke ();
    }
    reference6.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 7:
    GameObject reference7 = Instantiate (spawnListLevel7 [spotInspawnList], transform
        .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel7.Length) {
```

```
        CancelInvoke ();
    }
    reference7.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 8:
    GameObject reference8 = Instantiate (spawnListLevel8 [spotInspawnList], transform
    .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel8.Length) {
        CancelInvoke ();
    }
    reference8.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 9:
    GameObject reference9 = Instantiate (spawnListLevel9 [spotInspawnList], transform
    .position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel9.Length) {
        CancelInvoke ();
    }
    reference9.SendMessage ("setTheMascotWay", pathPoints);
    break;
case 10:
    GameObject reference10 = Instantiate (spawnListLevel10 [spotInspawnList],
    transform.position, Quaternion.identity) as GameObject;
    spotInspawnList++;
    if (spotInspawnList >= spawnListLevel10.Length) {
        CancelInvoke ();
    }
    reference10.SendMessage ("setTheMascotWay", pathPoints);
    break;
}
}
```

```
using UnityEngine;
using System.Collections;

public class MoneyTracker : MonoBehaviour {

    public int startMoney = 1500;
    public GameObject Moneytext;

    public static float money=0;

    void Start ()
    {
        money = startMoney;
    }
    void Update()
    {
        Moneytext.GetComponent<TextMesh>().text = money.ToString();
    }
}
```

```
using UnityEngine;
using System.Collections;

public class OnenemyDestoy : MonoBehaviour {

    //when the enemy is destroyed the amount they are worth is
    //added to the the player's money
    public float addMoney = 0.0f;
    void OnDestroy() {
        MoneyTracker.money += addMoney;
    }
}
```

```
using UnityEngine;
using System.Collections;

public class SelectButtonByClick : MonoBehaviour {

    // Use this for initialization
    public GameObject ShowButtonSelectandBackButton;//has to be main camera because that's
    where the code is
    public GameObject CreateTowerWhenclick;//same but code is on plane
    //public GameObject PausePlayButton;

    void Clicked()
    {
        //Tell the tower select to change the selected button
        //and create tower what tower was selected
        ShowButtonSelectandBackButton.SendMessage("playerChosentower", gameObject);
        CreateTowerWhenclick.SendMessage ("getSelectedTower", gameObject);
    }
}
```

```
using UnityEngine;
```

```
using System.Collections;
```

```
public class ShowButtonSelectandBackButton : MonoBehaviour {

    public GameObject tower1icon;
    public GameObject tower2icon;
    public GameObject tower3icon;
    public GameObject playButton;//the play button
    public GameObject BackButton;// the back button

    public GameObject Backbuttonobject;//the object with the code on it
    public GameObject PlayButtonobject;//the object with the code on it
    public float towerrotaion=1.0f;

    public int spot=0;

    private GameObject selectedTower;
    void Update(){
        if (selectedTower != null) {
            //selectedTower will rotate
            selectedTower.transform.Rotate (Vector3.left, towerrotaion * Time.deltaTime);
        }
    }

    void playerChosentower(GameObject clickedIcon){
        //setting selected icon to moving icon to show selection
        if (tower1icon == clickedIcon) {
            selectedTower=tower1icon;
        }
        if (tower2icon == clickedIcon) {
            selectedTower=tower2icon;
        }
        if (tower3icon == clickedIcon) {
            selectedTower=tower3icon;
        }
        if (playButton == clickedIcon) {
            if (MascotSpawner.Play==false){

                PlayButtonobject.SendMessage("BeginTheRelease", true);
                selectedTower=null;
            }
        }
        if (BackButton == clickedIcon) {
            MascotSpawner.Play=false;//so that when the player leaves the level the level's
            spawner is reset
            Backbuttonobject.SendMessage("changeToDifferentScene", "LevelSelect");
        }
    }
}
```

```
using UnityEngine;
using System.Collections;

public class TheRefsflag : MonoBehaviour {

    void OnCollisionEnter(Collision hitwithflag)
    {
        if (hitwithflag.collider.tag == "enemy") { //if the thing the flag hit was an enemy
            EnemyMovement getMascotsSpeed=hitwithflag.collider.GetComponent<EnemyMovement>();
            //if the enemy's speed has yet to be changed
            if(getMascotsSpeed.speed==getMascotsSpeed.originalspeed){
                getMascotsSpeed.speed= (getMascotsSpeed.speed* 0.5f);
            }
        }
    }
}
```